

Módulo 7: DevSecOps y Cierre

T20: Pentesting Paso a Paso

Carlos Rodríguez Contreras · GitHub: karrocon

 *OWASP ZAP, Burp Community y tu primer informe de auditoría*

Objetivos

- Entender el proceso de pentesting web: reconocimiento → escaneo → explotación → reporte
- Usar OWASP ZAP para escanear VulnApp de forma automatizada
- Usar Burp Suite Community para interceptar y manipular peticiones manualmente
- Escribir un informe de auditoría con hallazgos, severidad y recomendaciones

¿Qué es un Pentest?

Penetration Test — simulación controlada de un ataque real para encontrar vulnerabilidades.

Reconocimiento → Escaneo → Explotación → Post-explotación → Reporte

Fase	Actividad	Herramientas
Reconocimiento	Mapear superficie de ataque	nmap, OSINT, whois
Escaneo	Detectar vulnerabilidades	OWASP ZAP, nikto, Burp
Explotación	Probar las vulnerabilidades	curl, scripts custom, sqlmap
Post-explotación	Evaluar impacto real	Lateral movement, data access
Reporte	Documentar hallazgos	Plantilla de informe

OWASP ZAP – escaneo automatizado

```
# 1. Arrancar ZAP en modo headless (para CI) o GUI
docker run -u zap -p 8080:8080 owasp/zap2docker-stable zap.sh -daemon

# 2. Spider (descubrir URLs)
zap-cli spider http://localhost:3000

# 3. Active Scan (probar vulnerabilidades)
zap-cli active-scan http://localhost:3000

# 4. Ver alertas
zap-cli alerts
```

En la GUI:

1. URL objetivo → `http://localhost:3000`
2. Spider automático (descubre endpoints)
3. Active Scan (prueba SQLi, XSS, etc.)
4. Revisa alertas por severidad

Burp Suite Community – interceptación manual

Flujo de trabajo:

1. Configura Burp como proxy (127.0.0.1:8080)
2. Navega por la app → Burp captura cada petición
3. Envía peticiones al **Repeater** para modificarlas
4. Prueba payloads manualmente (SQLi, XSS, IDOR)
5. Documenta hallazgos

💡 ZAP para escaneo automático amplio. Burp para investigación manual profunda. Ambos se complementan.

Estructura del informe de auditoría

```
# Informe de Auditoría de Seguridad – VulnMotors

## Resumen Ejecutivo
- Alcance, fechas, metodología

## Hallazgos
| # | Vulnerabilidad | Severidad | CVSS | Estado |
|---|-----|-----|-----|-----|
| 1 | SQL Injection en login | CRITICAL | 9.8 | Corregido |
| 2 | XSS almacenado | HIGH | 8.1 | Corregido |
| ... | ... | ... | ... | ... |

## Detalle por hallazgo
### 1. SQL Injection en login
- **Descripción:** ...
- **Evidencia:** (captura / curl command)
- **Impacto:** ...
- **Fix aplicado:** ...
- **Verificación:** ...

## Recomendaciones generales
```

CVSS – cómo asignar severidad

Factor	Valores
Attack Vector	Network > Adjacent > Local > Physical
Attack Complexity	Low > High
Privileges Required	None > Low > High
User Interaction	None > Required
Impact (CIA)	High > Low > None

Ejemplos de nuestro curso:

Vulnerabilidad	CVSS	Justificación
SQLi en login	9.8	Network, Low complexity, No auth, High CIA
XSS almacenado	8.1	Network, Low complexity, No auth, High C
IDOR en perfil	6.5	Network, Low complexity, Low priv, High C

Responsible Disclosure

Si encuentras vulnerabilidades en sistemas reales:

1.  **NO** explotar más allá de lo necesario para confirmar
2.  Contactar al responsable (security@dominio, programa de bug bounty)
3.  Dar plazo razonable para corregir (90 días es estándar)
4.  Documentar todo el proceso
5.  No publicar hasta que esté corregido (o venza el plazo)

 Acceder a sistemas sin autorización es **delito penal** en la mayoría de jurisdicciones, independientemente de la intención.

DAST en CI/CD – pentesting automatizado

```
# .github/workflows/dast.yml
name: DAST with OWASP ZAP
on:
  workflow_run:
    workflows: ["Deploy to Staging"]
    types: [completed]

jobs:
  zap-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: zaproxy/action-full-scan@v0.10.0
        with:
          target: 'https://staging.vulnapp.example.com'
          rules_file_name: '.zap/rules.tsv'
          cmd_options: '-a'
      - uses: actions/upload-artifact@v4
        with:
          name: zap-report
          path: report_html.html
```

- ✓ Ejecutar ZAP automáticamente contra staging después de cada deploy detecta regresiones de seguridad antes de llegar a producción.

Metodología OWASP Testing Guide (resumen)

Fase	Tests clave
Information Gathering	Fingerprint servidor, enumerar endpoints, review JS
Configuration	Headers, TLS, error handling, métodos HTTP
Identity Management	Enumeración de usuarios, política de passwords
Authentication	Bypass, brute force, credential stuffing
Authorization	IDOR, privilege escalation, path traversal
Session Management	Cookie flags, fixation, timeout
Input Validation	SQLi, XSS, CMDi, SSRF (todos los inyección)
Error Handling	Stack traces, información sensible en errores
Cryptography	TLS config, almacenamiento de datos sensibles
Business Logic	Race conditions, flujos abusables

Lab T20: Pentest completo de VulnApp con ZAP

Objetivo: realizar un pentest básico de VulnApp y documentar los hallazgos

Setup: OWASP ZAP + VulnApp corriendo en `http://localhost:3000`

1. Lanza OWASP ZAP y configura el spider sobre `http://localhost:3000`
2. Ejecuta el Active Scan y espera a los resultados
3. Revisa los alertas: clasifícalos por severidad (High / Medium / Low)
4. Documenta 5 hallazgos en la plantilla de informe de auditoría del curso

 **Descargas gratuitas:** [OWASP ZAP](#) · [Burp Suite Community](#)